

Semantic segmentation with U-net

Beatriz Marcotegui

Centre de Morphologie Mathématique

MINES ParisTech

PSL Research University

beatriz.marcotegui@minesparis.psl.eu

September 2025

Plan

- 1 Introduction
- 2 Metrics
- 3 Optimizers
- 4 From classification to image-to-image translation
 - Receptive field
- 5 U-net architecture
- 6 Unet variants
- 7 Google Colab
- 8 Conclusions

Section 1

Introduction

Some vocabulary

- **Semantic segmentation** : each pixel gets a "class" label

Some vocabulary

- **Semantic segmentation** : each pixel gets a "class" label

original



Some vocabulary

- **Semantic segmentation** : each pixel gets a "class" label

original



semantic seg



Some vocabulary

- **Semantic segmentation** : each pixel gets a "class" label
- **Instance segmentation** : each pixel takes an "instance" label.

original



semantic seg



instance seg



Some vocabulary

- **Semantic segmentation** : each pixel gets a "class" label
- **Instance segmentation** : each pixel takes an "instance" label.
- **Panoptic segmentation** : each pixel takes "class" and "instance" labels.

original



semantic seg



instance seg



panoptic seg

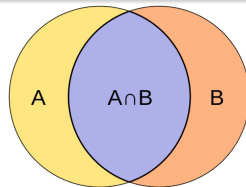


Section 2

Metrics

Measures used in image processing

Let A and B be two sets, not simultaneously empty
(Credits : public domain wikipedia.org.)



Jaccard index

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Dice coefficient

$$D(A, B) = \frac{2|A \cap B|}{|A| + |B|}$$

Properties

- $\forall A, B : 0 \leq J(A, B) \leq D(A, B) \leq 1$
- If $A = B$, then $D(A, B) = J(A, B) = 1$
- If $A \cap B = \emptyset$, then $D(A, B) = J(A, B) = 0$

Generalization to $[0, 1]$

Jaccard index

Also named Intersection over Union (IoU)

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

But $\mathbf{y}$ and $\hat{\mathbf{y}}$ are in $[0, 1]^n \dots$

Generalization to $[0, 1]$

Jaccard index

Also named Intersection over Union (IoU)

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

But $\mathbf{y}$ and $\hat{\mathbf{y}}$ are in $[0, 1]^n \dots$

Jaccard similarity

$$J(\mathbf{y}, \hat{\mathbf{y}}) = \frac{\sum_i y_i \hat{y}_i}{\sum_i y_i + \sum_i \hat{y}_i - \sum_i y_i \hat{y}_i}$$

($\mathbf{y}$ and $\hat{\mathbf{y}}$ are not simultaneously equal to 0)

Corresponding loss function

Jaccard loss

$$j(\mathbf{y}, \hat{\mathbf{y}}) = 1 - \frac{\sum_i y_i \hat{y}_i}{\sum_i y_i + \sum_i \hat{y}_i - \sum_i y_i \hat{y}_i + \epsilon}$$

Constant ϵ , which is typically “small”, keeps the denominator “far enough” from zero.

Jaccard Loss code (keras)

```
def jaccard1_coef(y_true, y_pred, smooth=SMOOTH):  
    y_true_f = K.flatten(y_true)  
    y_pred_f = K.flatten(y_pred)  
    intersection = K.sum(y_true_f * y_pred_f)  
    union = K.sum(y_true_f) + K.sum(y_pred_f) - intersection  
    return (intersection + smooth) / (union + smooth)  
  
def jaccard1_loss(y_true, y_pred, smooth=SMOOTH):  
    return 1 - jaccard1_coef(y_true, y_pred, smooth=SMOOTH)
```

Jaccard loss function - variant

Jaccard2 loss

$$j_2(\mathbf{y}, \hat{\mathbf{y}}) = 1 - \frac{\sum_i y_i \hat{y}_i}{\sum_i y_i^2 + \sum_i \hat{y}_i^2 - \sum_i y_i \hat{y}_i + \epsilon}$$

This version works slightly better than standard Jaccard loss (j_1).

- Cha, S. H. (2007). Comprehensive survey on distance/similarity measures between probability density functions. City, 1(2), 1.
- Duque-Arias, D. et al. (2021). On power Jaccard losses for semantic segmentation. In VISAPP 2021.

Loss functions

- Cross-entropy : $-\sum_i y_i \log(\hat{y}_i)$

Cross entropy

Classification loss function.

It is the sum of the true probability (y_i) multiplied by the log of the prediction ($\hat{y}_i$) over all classes of the distribution.

$$\text{Cross-entropy : } CE = - \sum_i y_i \log(\hat{y}_i)$$

input image $\Rightarrow$ MODEL $\Rightarrow$ vector of probabilities



	dog	cat	elephant	snike
True probability	1	0	0	0
Predicted prob.	0.91	0.08	0.01	0

Usually softmax is found at the end of DL classification systems, in order to convert output scores in probabilities (between $[0,1]$ and sum to 1).

Softmax operator

Definition

The softmax operator $\sigma : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is given by :

$$\forall \mathbf{x} \in \mathbb{R}^d, \forall k \in \{1, \dots, d\} : \quad \sigma(\mathbf{x})_k = \frac{e^{\mathbf{x}_k}}{\sum_{i=1}^d e^{\mathbf{x}_i}}$$

Softmax operator

Definition

The softmax operator $\sigma : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is given by :

$$\forall \mathbf{x} \in \mathbb{R}^d, \forall k \in \{1, \dots, d\} : \quad \sigma(\mathbf{x})_k = \frac{e^{\mathbf{x}_k}}{\sum_{i=1}^d e^{\mathbf{x}_i}}$$

Some properties

- $0 < \sigma(\mathbf{x})_k < 1$
- $\sum_{i=1}^d \sigma(\mathbf{x})_i = 1$

Softmax operator

Definition

The softmax operator $\sigma : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is given by :

$$\forall \mathbf{x} \in \mathbb{R}^d, \forall k \in \{1, \dots, d\} : \quad \sigma(\mathbf{x})_k = \frac{e^{\mathbf{x}_k}}{\sum_{i=1}^d e^{\mathbf{x}_i}}$$

Some properties

- $0 < \sigma(\mathbf{x})_k < 1$
- $\sum_{i=1}^d \sigma(\mathbf{x})_i = 1$

Example

$$\mathbf{x} = \begin{pmatrix} 10.1 \\ 0 \\ -4.3 \\ 1.33 \end{pmatrix} \quad \sigma(\mathbf{x}) \approx \begin{pmatrix} 0.9998 \\ 0.000041 \\ 0.00000056 \\ 0.00016 \end{pmatrix}$$

Cross entropy

Kullback - Leibler divergence

The dissimilarity between two probability distributions is estimated by the KL divergence.

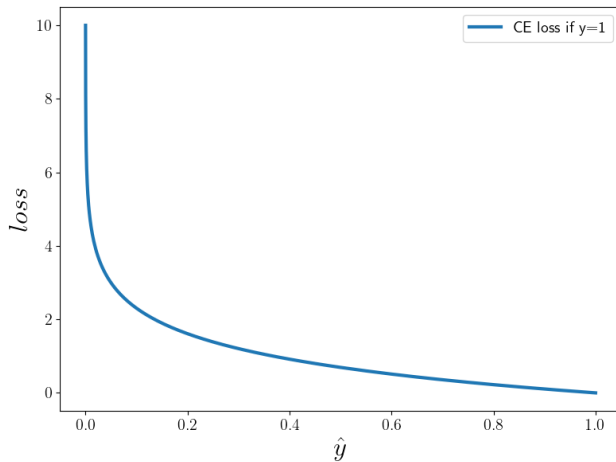
Consider two probability distributions P and Q : the true probability ($P = y$) and the predicted probability ($Q = \hat{y}_i$).

$$D_{KL}(P, Q) = \sum_i P_i \log \frac{P_i}{Q_i} = \sum_i P_i [\log(P_i) - \log(Q_i)] = \sum_i P_i \log(P_i) - \sum_i P_i \log(Q_i)$$

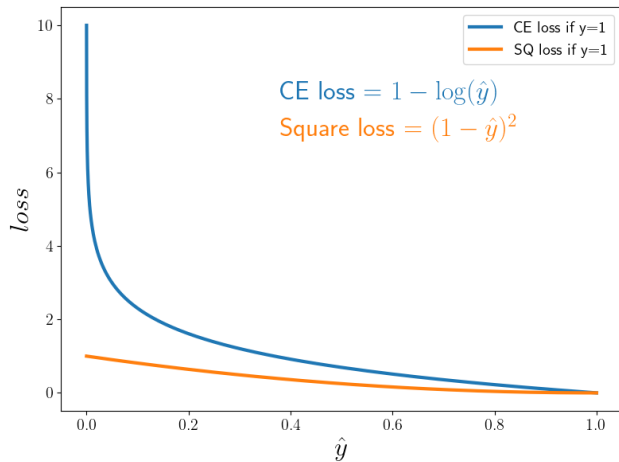
The first term $\sum_i P_i \log(P_i)$ is the entropy of the ground truth labels, that does not depend on θ . Therefore :

$$\operatorname{argmin}_{\theta} [D_{KL}(P, Q)] = \operatorname{argmin}_{\theta} [-\sum_i P_i \log(Q_i)] \Rightarrow \text{Cross Entropy formula}$$

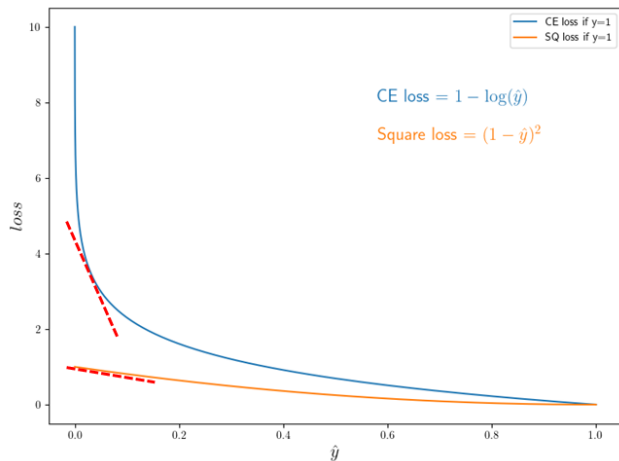
Cross entropy



Cross entropy

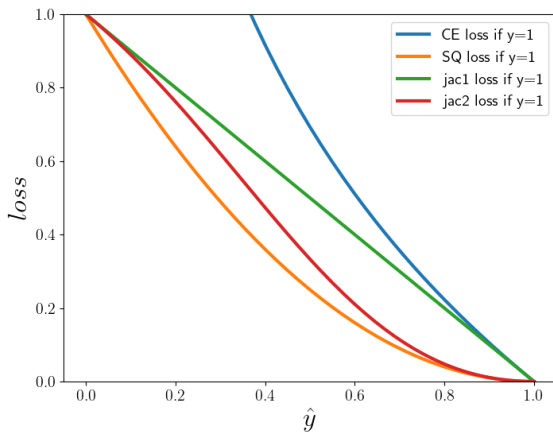


Cross entropy



Cross entropy : higher gradient for worst predictions. $\Rightarrow$ faster convergence

Some losses comparison



Sometimes, several losses are combined.

Focal loss

Goal

Designed to fight against class imbalance, focuses training on a sparse set of "hard" examples and prevents the vast number of "easy" samples from overwhelming the detector.

$$CE(p, y) = \begin{cases} -\log(p) & \text{if } y = 1 \\ -\log(1 - p) & \text{otherwise} \end{cases}$$

y : ground truth class

$p \in [0, 1]$: estimated probability for the class $y = 1$

$$p_t = \begin{cases} p & \text{if } y = 1 \\ (1 - p) & \text{otherwise} \end{cases}$$

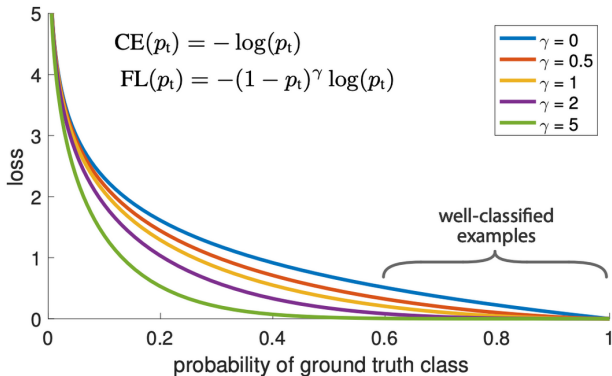
$$CE(p, y) = CE(p_t) = -\log(p_t)$$

Focal Loss for Dense Object Detection <https://arxiv.org/pdf/1708.02002v2.pdf>

Focal loss

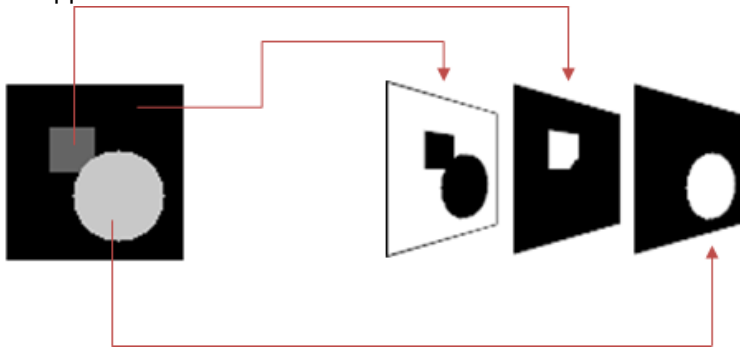
definition

$$FL(p_t) = -(1 - p_t)^\gamma \log(p_t)$$



Multi-class segmentation

Each class is set in a different output channel (one-hot encoding). Then binary segmentation is applied.



Multi-class segmentation

Each class is set in a different output channel.

For example :

1
0
2
2
1
0
2
2



0	1	0
1	0	0
0	0	1
0	0	1
0	1	0
1	0	0
0	0	1
0	0	1

$Y2 = tf.keras.utils.to_categorical(Y)$

multi-class segmentation metric

mean Intersection over Union (mIoU)

Usual multi-class segmentation metric :

$$mIoU = \frac{1}{N} \sum_{i=1}^N IoU_i$$

N : number of classes, IoU_i : IoU of class i .

Section 3

Optimizers

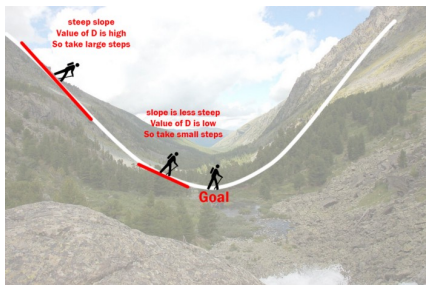
Optimizers

Method used to update the NN weights so that the loss in the training set is reduced. The problem is strongly non-convex. We do not have convergence guarantees. Only a few guidelines.

- Gradient descent
- Stochastic Gradient Descent (SGD)

Gradient descent

```
for t in nb_steps:  
    dw = compute_gradient(lossF, training_samples, w)  
    w = w - lr * dw
```



From <https://towardsdatascience.com/linear-regression-using-gradient-descent-97a6c8700931>

$$dw = \nabla_{\theta} \sum_i^N L(f(x_i, \theta_{t-1}), y_i)$$

with (x_i, y_i) , the training set and N the number of training samples.

Stochastic Gradient Descent

```
for t in nb_steps:  
    SelectBatch  
    dw = compute_gradient(lossF, batch, w)  
    w = w - lr * dw
```

$$dw = \nabla_{\theta} \sum_i^m L(f(x_i, \theta_{t-1}), y_i)$$

with (x_i, y_i) , the training set and m the number of batch samples.

Hyper-parameters

- weight initialization
- learning rate
- number of iterations
- batch size (stochastic!) because it is computationally expensive to compute the loss on the whole dataset

SGD + momentum

Momentum

Accelerates optimization, integrating gradient with an exponentially decaying moving average

```
v = 0
for i in nb_steps:
    dw = compute_gradient(lossF,data,w)
    v = rho * v + dw
    w = w - lr * v
```

- Intuition : a ball going down a hill. The ball does not follow the gradient direction because of its velocity.
- ρ typically 0.9 or 0.99

SGD + momentum

$$v_i = \rho v_{i-1} + dw_i$$

$$v_0 = dw_0$$

$$v_1 = \rho * dw_0 + dw_1$$

$$v_2 = \rho * v_1 + dw_2 = \rho(\rho * dw_0 + dw_1) + dw_2 = \rho^2 dw_0 + \rho dw_1 + dw_2$$

...

$$v_i = \rho^i dw_0 + \rho^{i-1} dw_1 + \dots + dw_i$$

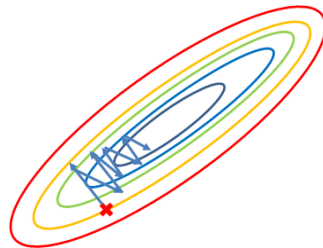
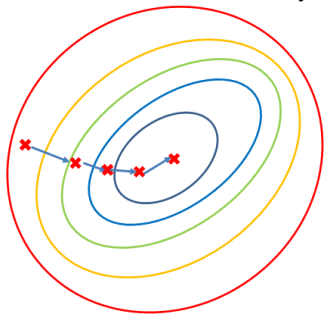
$$v_i = \rho^i dw_0 + \rho^{i-1} dw_1 + \dots + dw_i$$

- If gradient is in the same direction it is amplified
- Abrupt changes are smoothed

Sutskever et al."On the importance of initialization and momentum in DL" ICML 2013.

SGD difficulties

- If the gradient is high in one dimension and low in another one :
 - The optimization process may oscillate ("zigzag") if the learning rate is large
 - The optimization process may be slow, if the learning rate is small
- In both cases, too many iterations are required for convergence, if it converges !



SGD difficulties

- If the gradient is high in one dimension and low in another one :
 - The optimization process may oscillate ("zigzag") if the learning rate is large
 - The optimization process may be slow, if the learning rate is smallIn both cases, too many iterations are required for convergence, if it converges !
- local minima

SGD difficulties

- If the gradient is high in one dimension and low in another one :
 - The optimization process may oscillate ("zigzag") if the learning rate is large
 - The optimization process may be slow, if the learning rate is smallIn both cases, too many iterations are required for convergence, if it converges!
- local minima
- saddle points : increases in one dimension and decreases in another one

SGD difficulties

- If the gradient is high in one dimension and low in another one :
 - The optimization process may oscillate ("zigzag") if the learning rate is large
 - The optimization process may be slow, if the learning rate is smallIn both cases, too many iterations are required for convergence, if it converges!
- local minima
- saddle points : increases in one dimension and decreases in another one
- noisy "stochastic" estimations

AdaGrad

Adaptive learning rates (per parameter).

```
grad_sq = 0
for i in nb_steps:
    dw = compute_gradient(lossF,data,w)
    grad_sq += dw * dw
    w = w - lr*dw/(grad_sq+epsilon)
```

The squared gradient is always positive. Its sum is increasing over time. Then, AdaGrad decreases the learning rate over time.

The square gradient may become too big. The optimization progress may stop. Duchi et al. "Adaptive subgradient methods for online learning and stochastic optimization" JMLR 2011.

RMSProp : "Leak Adagrad"

```
grad_sq = 0
for i in nb_steps:
    dw = compute_gradient(lossF,data,w)
    grad_sq = decay_rate * grad_sq + (1-decay_rate) * dw * dw
    w = w - lr*dw/(grad_sq+epsilon)
```

Intuition : do not slow down over the optimization process.

Momentum based optimization can overshoot and then come back.

AdaGrad and RMSProp arrest progress along the fast moving direction while simultaneously accelerating progress along slow moving direction. $\Rightarrow$ Bends directly to the minimum.

Adam : RMSProp + momentum

Bias at the beginning of the optimization process, gradient is close to 0.

```
grad_sq = 0
for i in nb_steps:
    dw = compute_gradient(lossF,data,w)
    moment1 = beta1 * moment1 + (1-beta1) * dw
    moment2 = beta2 * moment2 + (1-beta2) * dw * dw

    momment1_unbias = moment1/(1-beta1**i)
    momment2_unbias = moment2/(1-beta2**i)
    w = w - lr*moment1_unbias/(moment2_unbias+epsilon)
```

default values : $\beta_1 = 0.9$; $\beta_2 = 0.999$

Kingma and Ba, "Adam : A method for stochastic optimization", ICLR 2015.

Weigth Decay

Idea : keep weights small unless absolutely necessary. A regularization technique that prevents overfitting.

$$\mathcal{L}_{total} = \mathcal{L}_{original} + \lambda \sum w_i^2$$

- $\mathcal{L}_{original}$: Original loss, i.e. cross entropy, dice...
- $\sum w_i^2$: L_2 norm of all weights
- λ : weight decay coefficient

AdamW

Adam : weight decay added as L2 regularization term to loss function.

AdamW : weight decay applied directly to weight updates

$$w = w - lr \left(\text{moment1_unbias} / (\text{moment2_unbias} + \text{epsilon}) + \text{lambda } w \right)$$

	Adam	AdamW
Weight Decay	L2 regularization applied to the loss	Applied directly to weight updates
Impact	Can interfere with adaptive lr	Decoupled, leads to consistent updates
Generalization	Can cause overfitting in complex models	Better regularization and generalization

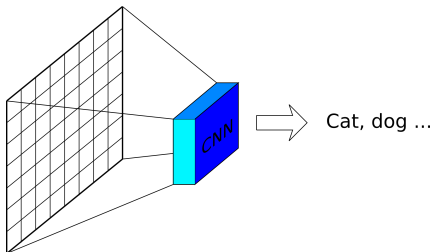
Adam vs AdamW

- **Adam** : may converge faster but can suffer from poor generalization
- **AdamW** : in case of large models, AdamW may help to achieve better generalization and stable convergence.

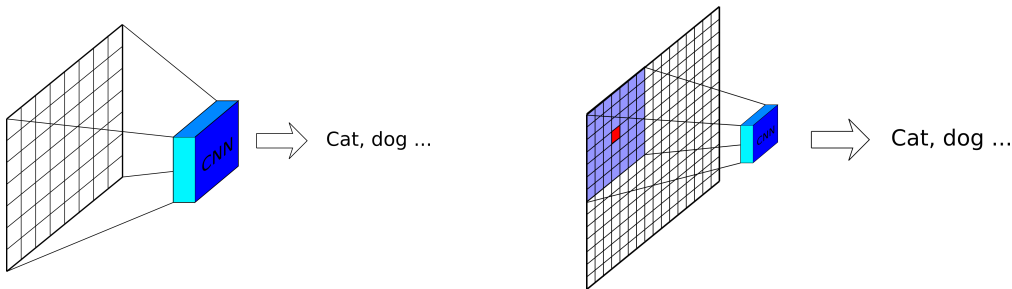
Section 4

From classification to image-to-image translation

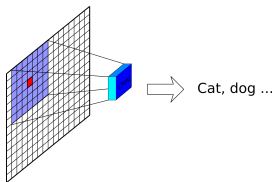
From classification to image-to-image translation



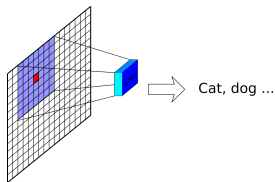
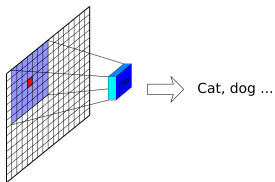
From classification to image-to-image translation



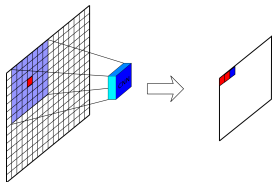
From classification nets to image-to-image nets



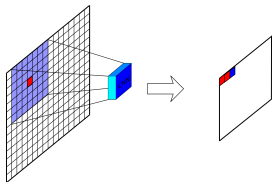
From classification nets to image-to-image nets



From classification nets to image-to-image nets



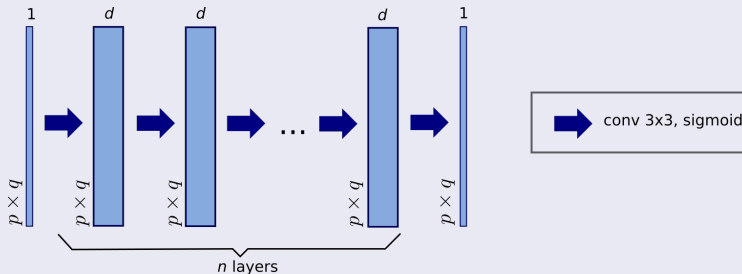
From classification nets to image-to-image nets



- The neuron membrane segmentation challenge winner [?] used this strategy. It is inefficient.
- The idea to compute the whole output image with a single pass through the network had already been proposed [?, ?] (but with architectures more complex than those used today).

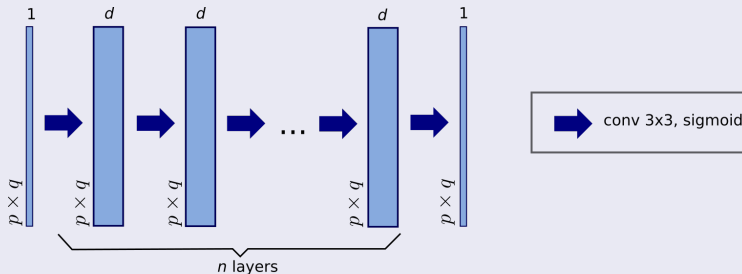
The simplest image-to-image architecture

Example : plain CNN [?]



The simplest image-to-image architecture

Example : plain CNN [?]

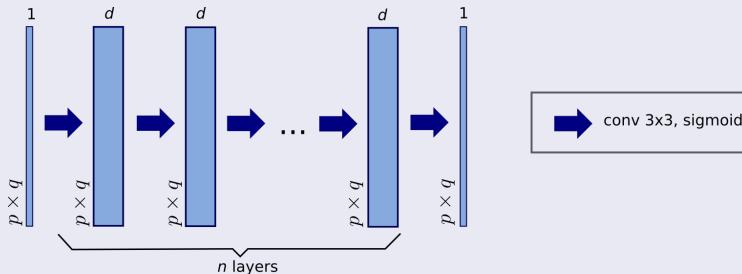


Note that today we would use ReLU activations on all layers except the last.

How many parameters does the first layer contain ? The last ? The second ?

The simplest image-to-image architecture

Example : plain CNN [?]

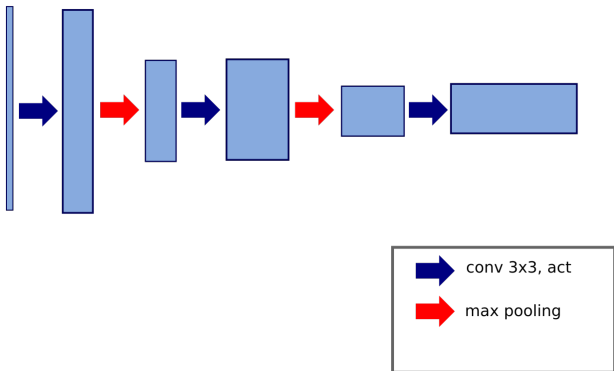


Note that today we would use ReLU activations on all layers except the last.

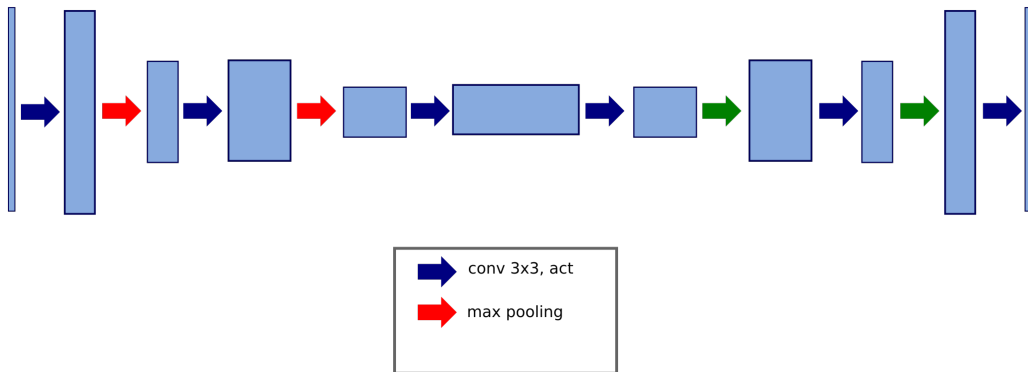
How many parameters does the first layer contain ? The last ? The second ?

$$(9 + 1) \times d ; 9d + 1 ; (9d + 1) \times d$$

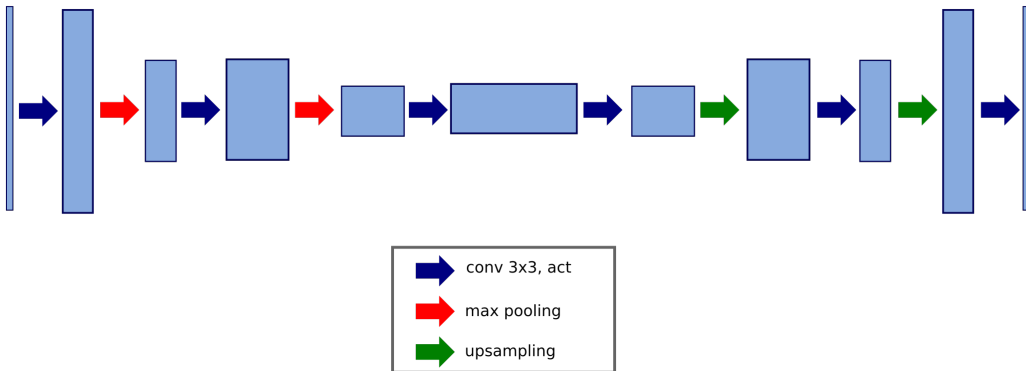
Going back to the original image size



Going back to the original image size



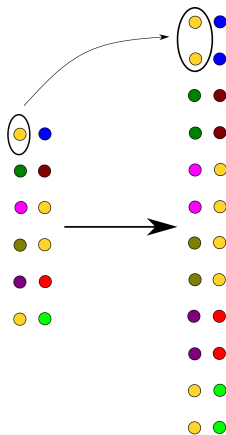
Going back to the original image size



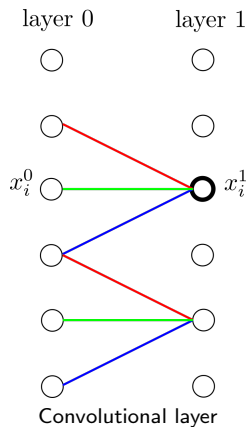
Upsampling techniques

- Replication
- Transposed convolution
- Pooling index memorization

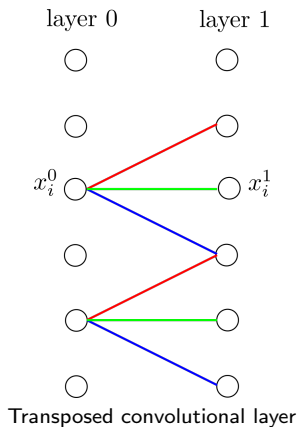
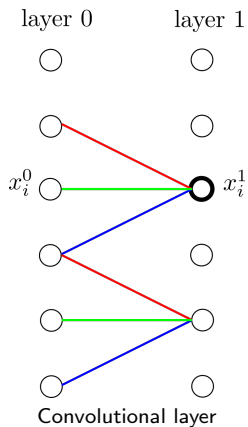
Upsampling through replication



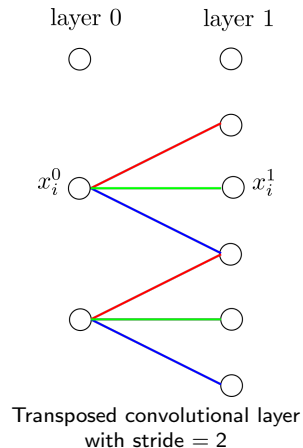
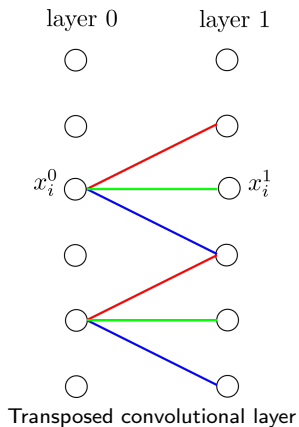
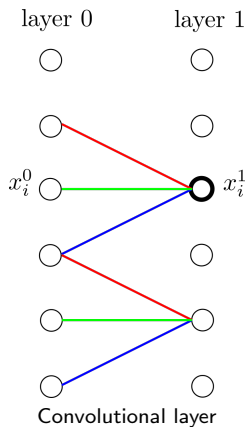
Transposed convolution



Transposed convolution



Transposed convolution



Sous-section 1

Receptive field

Receptive field

Definition : links between neurons

In a NN, we say that neuron a is linked to neuron b if there is an oriented path in the corresponding graph going from a to b .

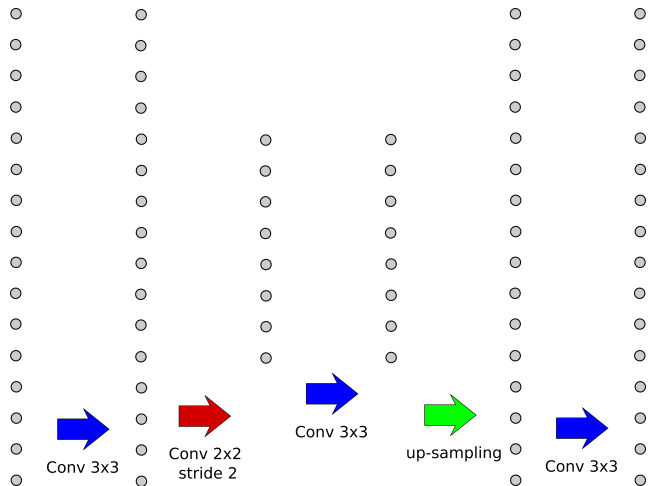
Definition

The **receptive field** of a neuron in a NN is the set of *input neurons* that are linked to that neuron.

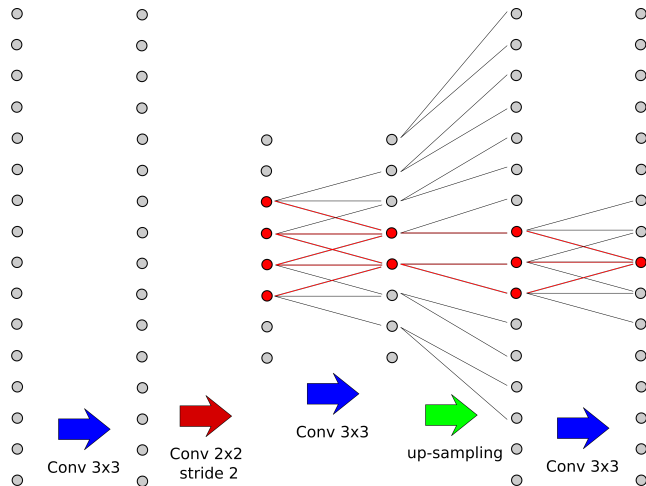
The size of the receptive field is an essential property when designing a fully-convolutional NN architecture.

Note that in most cases, receptive fields are square shaped (border effects ignored).

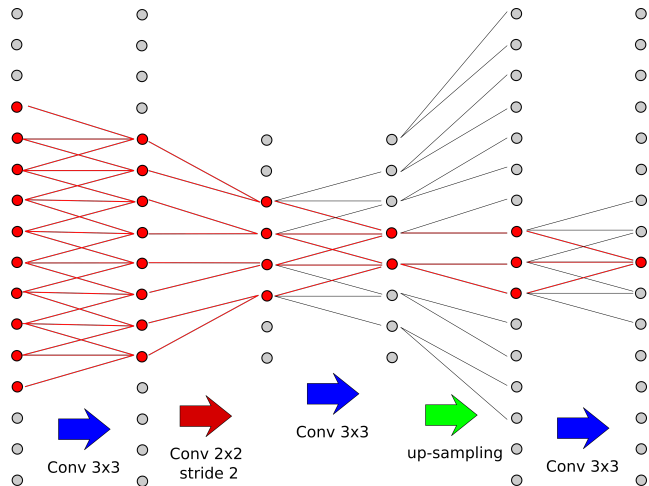
Illustration



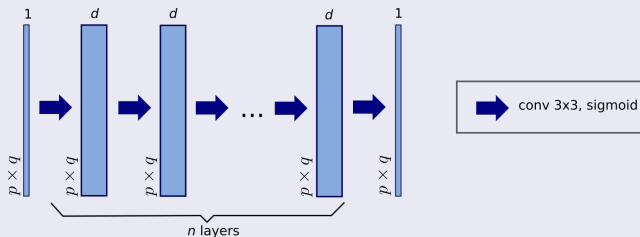
Illustration



Illustration



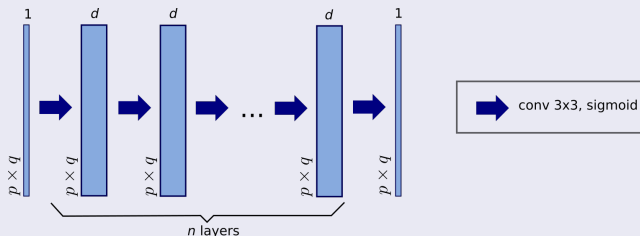
Example



What is the width of the receptive field of the neurons in the last layer?

- ① n
- ② $1 + 2 \times n$
- ③ $1 + 2 \times (n + 1)$
- ④ It depends on the input image

Example

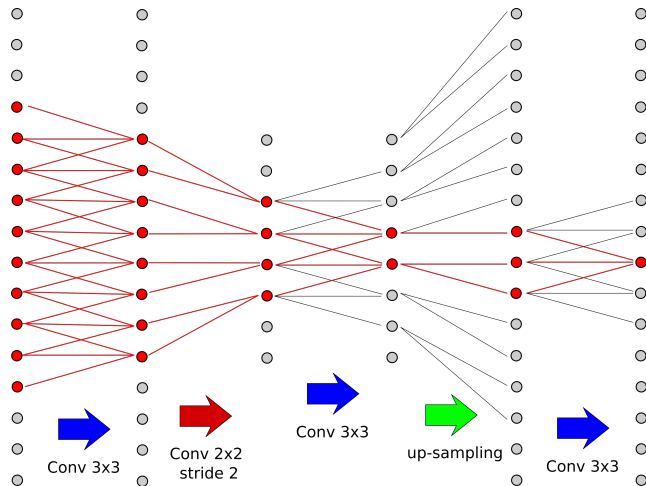


What is the width of the receptive field of the neurons in the last layer?

- ① n
- ② $1 + 2 \times n$
- ③ $1 + 2 \times (n + 1)$
- ④ It depends on the input image

Answer : $1 + 2 \times (n + 1)$

Effective receptive field : intuition



Conclusion on the receptive field

When choosing or designing a NN architecture :

- Establish its minimal required size with respect to the application
- Make it larger than that...

Section 5

U-net architecture

U-net architecture

Architecture

- Fully convolutional network, with an encoder and a decoder.
- The **encoder** (contracting path) is inspired by typical classification networks :
 - a series basic blocks : Convolution (3x3), Activation (RELU) and max-pooling 2x2 for downsampling.
 - At each downsampling step, the number of channels (features) is multiplied by 2.
- The **decoder** (expanding path)
 - upsampling the feature map
 - concatenate corresponding features from contracting path $\Rightarrow$ [Skip connection](#)
 - Basic block : Convolution (3x3), Activation (RELU)
- A final Convolution 1x1 with the output number of classes.

Ronneberger, O., Fischer, P., and Brox, T. (2015). U-net : Convolutional networks for biomedical image segmentation. MICCAI 2015. University of Freiburg, Germany.

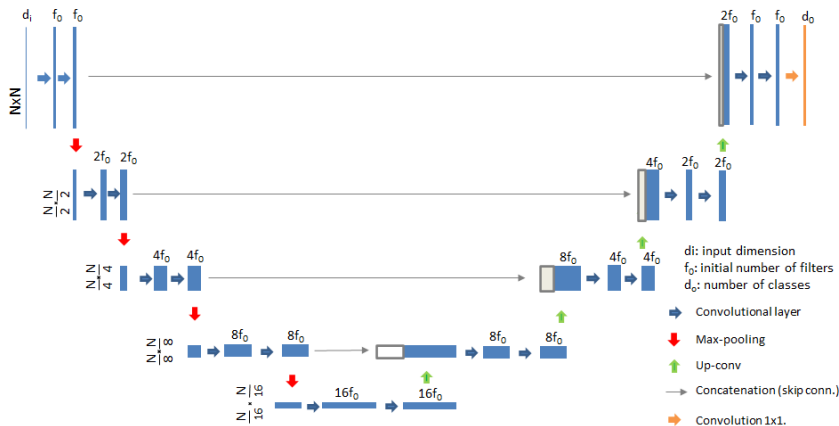
Skip connection

The key idea

Skip connection combine :

- spatial information from the encoding part
- with up-sampled deeper layers with more relevant feature representations.

U-net architecture



U-net implementation (keras)

```
def modular_u_net(shape, nb_levels = 4, nb_filters=32,output_channels=1):
    x = Input(shape); first_layer=[]; first_layer.append(x); skip_layers=[]
    # ENCODER
    for lev in range(nb_levels):
        x = unet_block(nb_filters, res, "conv"+str(lev+1), batch_norm)(x)
        if(lev < nb_levels -1):#not the last one
            skip_layers.append(x)
            x = unet_down(conv_sampling, nb_filters)(x)
            nb_filters = nb_filters * 2
    # DECODER
    for lev in range(nb_levels-1):
        x = unet_up(conv_sampling, nb_filters)(x)
        x = concatenate([x, skip_layers.pop()], axis=3)
        nb_filters= int(nb_filters/2)
        x = unet_block(nb_filters, res, "conv"+str(nb_levels+lev+1), batch_norm)(x)
    xFinal = Conv2D(output_channels, 1, activation="sigmoid", name="out")(x)
    return Model(first_layer.pop(), xFinal)
```

U-net block

```
def unet_block(nb_fil, res, name, batch_norm):  
def fn(tensor):  
    x = Conv2D(nb_fil, 3, padding="same", name="%s_1" % name)(tensor)  
    if batch_norm:  
        x = BatchNormalization()(x)  
    x = Activation("relu")(x)  
    x = Conv2D(nb_fil, 3, padding="same", name="%s_2" % name)(x)  
    if batch_norm:  
        x = BatchNormalization()(x)  
    if res:  
        skip=Conv2D(nb_fil,3,padding="same",name="%s_res"% name)(tensor)  
        x = add([x, skip])  
    x = Activation("relu")(x)  
    return x  
return fn
```

U-net down block

```
def unet_down(conv_sampling, nb_filters=None):  
    if conv_sampling:  
        if nb_filters is None:  
            raise ValueError("When conv_sampling is True, nb_filters should  
        def fn(tensor):  
            x = Conv2D(nb_filters, 3, padding="same", strides=(2, 2))(tensor)  
            return x  
    else:  
        def fn(tensor):  
            x = MaxPooling2D(pool_size=(2, 2))(tensor)  
            return x  
    return fn
```


U-net up block

```
def unet_up(conv_sampling, nb_filters):  
    if conv_sampling:  
        if nb_filters is None:  
            raise ValueError("When conv_sampling is True, nb_filters should  
        def fn(tensor):  
            x = Conv2DTranspose(nb_filters, 3, padding="same", strides=(2,  
            return x  
    else:  
        def fn(tensor):  
            x = UpSampling2D(size=(2, 2))(tensor)  
            return x  
    return fn
```

Section 6

Unet variants

Unet variants

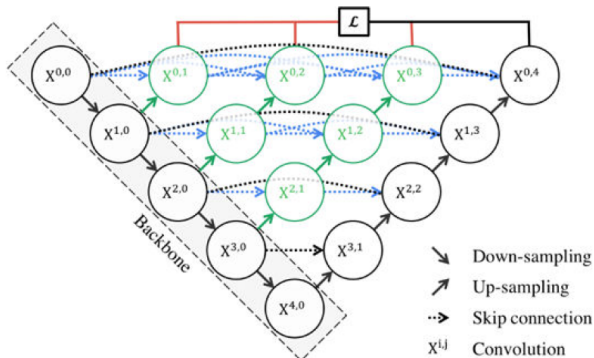
- Add residual connections in the convolutional blocks
- Unet ++ : A nested u-net architecture for medical image segmentation. 2018.
- Attention u-net
- UnetR : U-net Transformer
- Sharp U-net
- Motion U-net
- ...

Siddique, N., Paheding, S., Elkin, C. P., & Devabhaktuni, V. (2021). U-net and its variants for medical image segmentation : A review of theory and applications. IEEE Access, 9.

Unet++

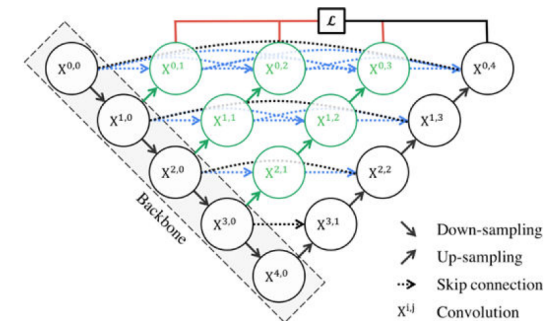
Dense skip pathways

- Aim : reduce semantic gap between the encoder and decoder feature maps.
- $x^{0,1} = H[x^{0,0}, U(x^{1,0})]$; $x^{0,2} = H[x^{0,0}, x^{0,1}, U(x^{1,1})]$;
 $x^{0,3} = H[x^{0,0}, x^{0,1}, x^{0,2}, U(x^{1,2})]$...

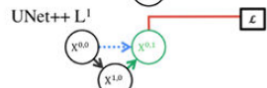
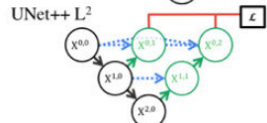
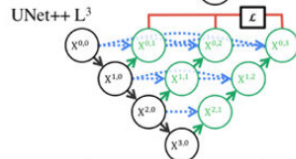
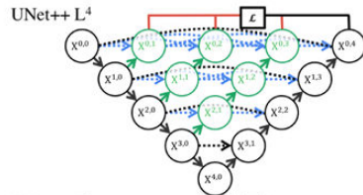
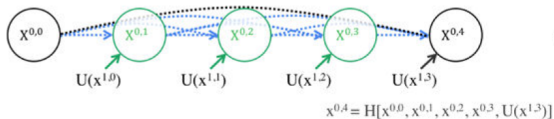


Zhou, Z. et al. (2018). Unet++ : A nested u-net architecture for medical image segmentation. In DL in medical image analysis and multimodal learning for clinical decision support (pp. 3-11).

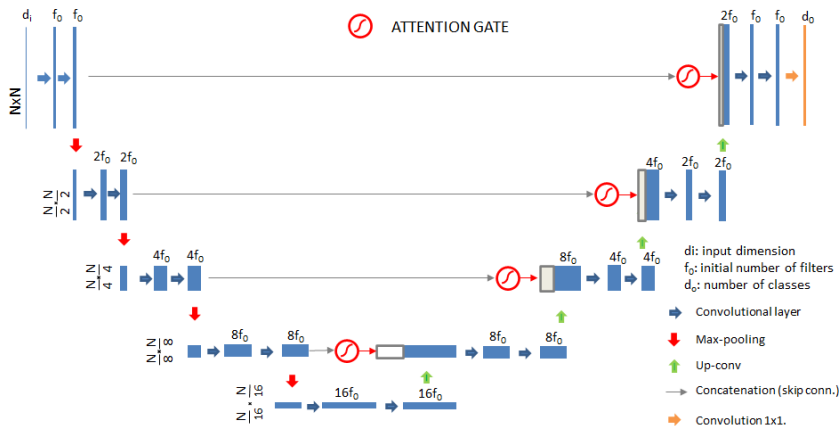
Unet++



$$x^{0,1} = H[x^{0,0}, U(x^{1,0})] \quad x^{0,2} = H[x^{0,0}, x^{0,1}, U(x^{1,1})] \quad x^{0,3} = H[x^{0,0}, x^{0,1}, x^{0,2}, U(x^{1,2})]$$



Attention U-net



Attention u-net

- Highlights relevant activations during training
- Reduces computational resources on irrelevant activations
- Better generalization

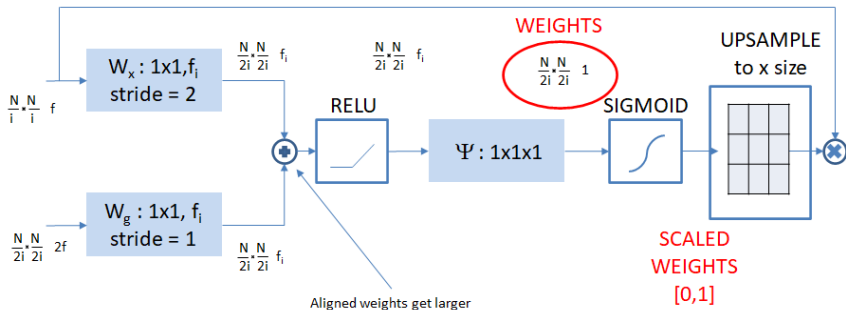
Soft attention

- Attention weights the relevance of different parts of the image
- The more relevant, the larger weight
- Backpropagation can be used
- The weights are trained
- Adds weights to pixels based on their relevance

Oktay, O., Schlemper, J., Folgoc, L. L., Lee, M., Heinrich, M., Misawa, K., ... & Rueckert, D. (2018). Attention u-net : Learning where to look for the pancreas. arXiv preprint arXiv :1804.03999.

Attention gate

- Takes two inputs :
 - x from the skip connection, with better spatial information
 - g from the lower layer, with better feature representation



Attention gate code

```
def attention_block_2d(x, g, inter_channel):  
    # x is the skip connexion  
    x2 = Conv2D(inter_channel, [1, 1], strides=[1, 1])(x)  
    # g is the up-sampled deeper level feature (stride is then 1)  
    g2 = Conv2D(inter_channel, [1, 1], strides=[1, 1])(g)  
  
    # add both branches and then activation  
    f = Activation('relu')(add([x2, g2]))  
  
    # get the scalar value, then apply sigmoid  
    psi_f = Conv2D(1, [1, 1], strides=[1, 1])(f)  
    rate = Activation('sigmoid')(psi_f)  
  
    # multiply  
    att_x = multiply([x, rate])  
    return att_x
```

Section 7

Google Colab

Google Colab

- Go to the website <https://colab.research.google.com/>
- Open a new notebook or an existing one
- For existing notebooks, upload it to your google drive
- Copy it in the colab content. For this purpose :
`!cp /content/gdrive/MyDrive/Colab Notebooks/my_notebook.ipynb /content"`
- Copy additional files :
`!cp /content/gdrive/MyDrive/Colab Notebooks/Unet_MAREVA/parameters.py /content`
`!cp -r /content/gdrive/MyDrive/Colab Notebooks/Unet_MAREVA/dl_tools /content`
Replace "Unet_MAREVA" with your directory name on google drive.
- In the first cell of your notebook add the following two lines :
`from google.colab import drive`
`drive.mount('/content/gdrive')`

Section 8

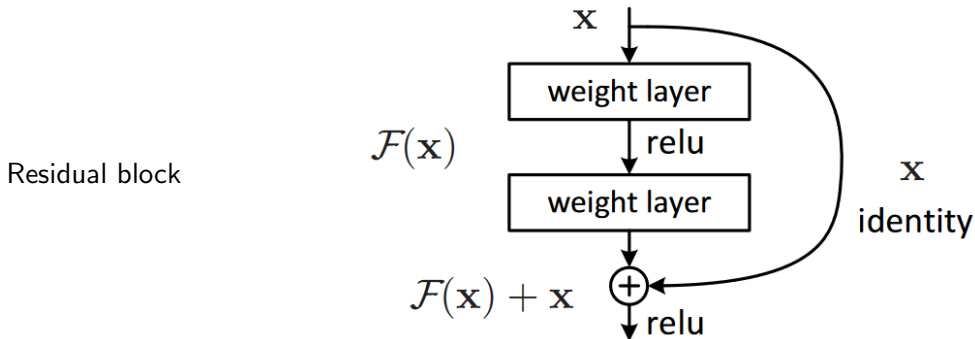
Conclusions

Conclusions

- U-net and its variants continue to be a prominent method for semantic segmentation
- For medical images and other domains
- Rmsprop or Adam can be used as default optimizers
- Jaccard2 or Cross-Entropy can be used as a baseline
- Loss is computed pixel-wise
- Working on specific losses enforcing structure is an interesting research topic

Residual learning (ResNet)

Shortcut connection : Allow deeper networks with less risk of performance loss.



He, Kaiming, et al. "Deep residual learning for image recognition." Proceedings of the IEEE conference on computer vision and pattern recognition. 2016.

https://openaccess.thecvf.com/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf

Google colab

- Upload files (notebook, dl_tools directory and parameters.py file) to your google drive
- mount your google drive
 - from google.colab import drive
 - drive.mount('/content/gdrive')
- change directory
 - import os
 - os.chdir("gdrive/My Drive/")

Section 9

References

References I